Quick tip: Review the prerequisites before you run the lab

Start Lab 01:30:00

Deploy Firebase Genkit Apps to Google Cloud

Lab 1 hour 30 minutes No cost Intermediate

★★★★☆

This lab may incorporate AI tools to support your learning.

Previous Next

Overview

Firebase Genkit is an open source framework that helps you build, deploy, and monitor production-ready AI-powered apps.

Genkit is designed for app developers. It helps you to easily integrate powerful AI capabilities into your apps with familiar patterns and paradigms.

Use **Genkit** to create apps that generate custom content, use semantic search, handle unstructured inputs, answer questions with your business data, autonomously make decisions, orchestrate tool calls, and much more.

Objective

This lab provides an introductory, hands-on experience with Firebase Genkit and the Gemini foundation model. You leverage the default flow and deploy it to both Google Cloud and Firebase.

You learn how to:

- Create a new Firebase Genkit project.
- Create flows with Genkit and Gemini.
- Deploy a Genkit flow to Cloud Run.
- Deploy a Genkit flow to Firebase.

Setup and requirements

Before you click the Start Lab button

Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

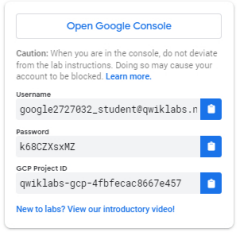
To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

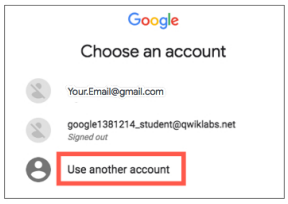
Note: If you are using a Pixelbook, open an Incognito window to run this lab.

1. Click the **Start Lab** button. If you need to pay for the lab, a pop-up opens for you to select your payment method. On the left is a panel populated with the temporary credentials that you must use for this lab.



2. Copy the username, and then click **Open Google Console**. The lab spins up resources, and then opens another tab that shows the **Choose an account** page.

Note: Open the tabs in separate windows, side-by-side.



4. Paste the username that you copied from the Connection Details panel. Then copy and paste the password.

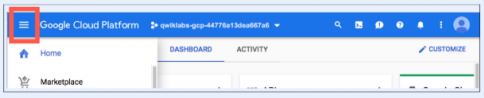
Note: You must use the credentials from the Connection Details panel. Do not use your Google Cloud Skills Boost credentials. If you have your own Google Cloud account, do not use it for this lab (avoids incurring charges).

5. Click through the subsequent pages:

- Accept the terms and conditions.
- Do not sign up for free trials.

After a few moments, the Cloud console opens in this tab.

Note: You can view the menu with a list of Google Cloud Products and Services by clicking the **Navigation menu** at the top-left.

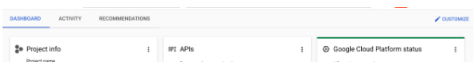


Activate Google Cloud Shell

Google Cloud Shell is a virtual machine that is loaded with development tools. It offers a persistent 5GB home directory and runs on the Google Cloud.

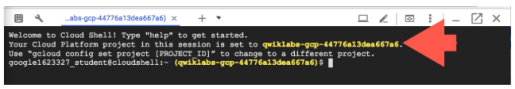
Google Cloud Shell provides command-line access to your Google Cloud resources.

1. In Cloud console, on the top right toolbar, click the Open Cloud Shell button



2. Click **Continue**.

It takes a few moments to provision and connect to the environment. When you are connected, you are already authenticated, and the project is set to your *PROJECT_ID*. For example:



gcloud is the command-line tool for Google Cloud. It comes pre-installed on Cloud

Shell and supports tab-completion.

- You can list the active account name with this command:

```
gcloud auth list
```

```
Credentialed accounts:
 - <myaccount>@<mydomain>.com (active)
</mydomain></myaccount>
```

Example output:

```
Credentialed accounts:
 - google1623327_student@qwiklabs.net
```

- You can list the project ID with this command:

```
gcloud config list project
```

Output:

```
[core]
project = <project_id>
</project_id>
```

```
[core]
project = qwiklabs-gcp-44776a13dea667a6
```

Note: Full documentation of **gcloud** is available in the gcloud CLI overview guide .

Task 1. Setup your Genkit Project

In this task, you set up a node application to use Firebase Genkit, and configure it to access the Gemini model in Google Cloud's Vertex AI.

Set up your Genkit application

following command:

```
export GCLOUD_PROJECT=Project_ID
export GCLOUD_LOCATION=Region
gcloud config set project Project_ID
gcloud services disable cloudfunctions.googleapis.com
gcloud services enable cloudfunctions.googleapis.com
```

The first three lines specify the active project and the region that should be used.

Disabling and enabling Cloud Run functions ensures that the service account used by Genkit for functions is available.

2. To authenticate to Google Cloud and set up credentials for your project and user, run the following command:

```
gcloud auth application-default login
```

- When prompted, type **Y**, and press **Enter**.
- To launch the Google Cloud sign-in flow in a new tab, press Control (for

- In the new tab, click the student email address.
- When you're prompted to continue, click **Continue**, and click **Allow**.
- To get the verification code, click **Allow**.
- Click **Copy**.
- Back in the terminal, where it says **Enter authorization code**, paste the copied verification code, and press **Enter**.

You are now authenticated to Google Cloud.

3. To create a directory for your project and initialize a new **Node** project, run the following commands:

```
mkdir genkit-intro && cd genkit-intro
npm init -y
```

This command creates a `package.json` file in the `genkit-intro` directory.

```
npm install -D genkit-cli@1.0.4 tsx typescript
```

5. To install the core **Genkit** package and other dependencies for your app, run the following commands:

```
npm install genkit@1.0.4 --save
npm install @genkit-ai/vertexai@1.0.4 @genkit-ai/google-
cloud@1.0.4 @genkit-ai/express@1.0.4 --save
```

Develop the Genkit application code

1. Create the application source folder and main file:

```
mkdir src && touch src/index.ts
```

2. To open the Cloud Shell editor, click **Open Editor** (✎) on the menu bar.

4. Open the `~/genkit-intro/package.json` file, and review all the dependencies that were added.

   The dependency list should look similar to this:

```
"dependencies": {
  "@genkit-ai/express": "^1.0.4",
  "@genkit-ai/google-cloud": "^1.0.4",
  "@genkit-ai/vertexai": "^1.0.4",
  "genkit": "^1.0.4"
},
```

5. Go to the `src` folder and open the `index.ts` file. Add the following import library references to the index.ts file:

```
import { z, genkit } from 'genkit';
import { vertexAI } from '@genkit-ai/vertexai';
import { gemini20Flash001 } from '@genkit-ai/vertexai';
import { logger } from 'genkit/logging';
import { enableGoogleCloudTelemetry } from '@genkit-
```

6. After the import statements, add code to initialize and configure the `genkit` instance, and configure logging and telemetry:

```
const ai = genkit({
    plugins: [
        // Load the Vertex AI plugin. You can optionally
specify your project ID
        // by passing in a config object; if you don't, the
Vertex AI plugin uses
        // the value from the GCLOUD_PROJECT environment
variable.
        vertexAI({ location: 'Lab Region' }),
    ],
});

logger.setLogLevel('debug');
enableGoogleCloudTelemetry();
```

7. Add code to define a flow named **menuSuggestionFlow** which prompts the model to suggest an item for a menu of a themed restaurant, with the theme

```
export const menuSuggestionFlow = ai.defineFlow(
    {
        name: 'menuSuggestionFlow',
        inputSchema: z.string(),
        outputSchema: z.string(),
    },
    async (subject) => {
        const llmResponse = await ai.generate({
            prompt: `Suggest an item for the menu of a
${subject} themed restaurant`,
            model: gemini20Flash001,
            config: {
                temperature: 1,
            },
        });
        return llmResponse.text;
```

```
    }
);
```

8. Add code to start the flow server and expose your flows as **HTTP** endpoints on the specified port:

```
    flows: [menuSuggestionFlow],
    port: 8080,
    cors: {
        origin: '*',
    },
});
```

Note: The CORS setting allows any origin, which is designed to simplify the development of this lab. You will want a more restrictive CORS policy for production apps.

Test the application

1. To test the application, click **Open Terminal**, and, in **Cloud Shell**, run the following commands:

```
cd ~/genkit-intro
npx genkit start --noui -- npx tsx src/index.ts
```

2. When prompted, press **Enter** to continue.

   Wait till the Genkit flow server starts. When ready, you should see output similar to:

```
Flow server running on http://localhost:8080
Reflection server (1769) running on http://localhost:3100
```

3. Open a new terminal in Cloud Shell, and run:

```
cd ~/genkit-intro
npx genkit flow:run menuSuggestionFlow '"French"' | tee
response.txt
```

   The result is shown.

4. You can also run a flow and stream the response:

5. To access the flow using a POST request, run:

```
curl -X POST "http://localhost:8080/menuSuggestionFlow" -H
"Content-Type: application/json"  -d '{"data": "French"}'
```

6. Return to the first **Cloud Shell Terminal**, and press **CTRL+C** to exit.

7. For lab assessment purposes, run the following commands to move the files to the Cloud Storage Bucket named `GCS Bucket Name`.

```
gsutil -m cp -r ~/genkit-intro/package.json gs://GCS Bucket
Name
gsutil -m cp -r ~/genkit-intro/src/index.ts gs://GCS Bucket
Name
gsutil -m cp -r ~/genkit-intro/response.txt gs://GCS Bucket
Name
```

   Click **Check my progress** to verify the objectives.

   Setup your Genkit Project

Note: Copying the files to the Google Cloud Storage location is used in the assessment process. If you've made file updates after copying the files, please upload the modified files again by running the same commands.

Task 2. Deploy the flow to Cloud Run

In this task, you deploy your **Genkit** application to Cloud Run, so that anyone can access it over the internet.

1. In Cloud Shell, to update `package.json`, run the following commands:

```
npm pkg set main="lib/index.js" scripts.build="tsc"
```

The `package.json` file now should look similar to:

```
{
    "name": "genkit-intro",
    "version": "1.0.0",
    "main": "lib/index.js",
    "scripts": {
        "test": "echo \"Error: no test specified\" && exit 1",
        "build": "tsc"
    },
    "keywords": [],
    "author": "",
    "license": "ISC",
    "type": "commonjs",
    "description": "",
    "devDependencies": {
        "genkit-cli": "^1.0.4",
        "tsx": "^4.19.3",
        "typescript": "^5.8.2"
    },
    "dependencies": {
        "@genkit-ai/express": "^1.0.4",
        "@genkit-ai/google-cloud": "^1.0.4",
        "@genkit-ai/vertexai": "^1.0.4",
        "genkit": "^1.0.4"
```

The exact versions of some libraries might be different from what is shown.

3. To create a default Typescript configuration file to build your application, in Cloud Shell, run the following command:

```
cat <<EOF >~/genkit-intro/tsconfig.json
{
    "compileOnSave": true,
    "include": [
        "src"
    ],
    "compilerOptions": {
        "module": "NodeNext",
        "noImplicitReturns": true,
        "outDir": "lib",
        "sourceMap": true,
        "strict": true,
        "target": "es2017",
        "skipLibCheck": true,
        "esModuleInterop": true,
        "moduleResolution": "nodenext",
        "noUnusedLocals": true
    }
}
EOF
```

A tsconfig.json file is a configuration file for TypeScript projects that specifies compiler options and settings that are needed to compile your project.

4. To run the build, in the **Cloud Shell**, run the following commands:

```
cd ~/genkit-intro
npm run build
```

5. To deploy the application to **Cloud Run**, run the following command:

```
gcloud run deploy Cloud Run Service Name --region Lab
Region --allow-unauthenticated --source .
```

6. When prompted to continue, type **Y**, and press **Enter**.

It may take a few minutes to deploy the service.

```
export SERVICE_URL=$(gcloud run services describe Cloud Run
Service Name --platform managed --region Lab Region --
format 'value(status.url)')
curl -X POST "$SERVICE_URL/menuSuggestionFlow" -H
"Authorization: Bearer $(gcloud auth print-identity-token)"
-H "Content-Type: application/json" -d '{"data": "banana"}'
```

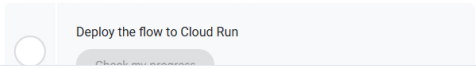
The first command retrieves the URL for the Cloud Run service, and the second command calls the flow hosted on the service.

8. In the Google Cloud Console, in the **Navigation menu** (☰), click **Cloud Run**, and click the `Cloud Run Service Name` service.

9. View the metrics for the call that you just performed.

10. To see the logs for your service, select the **Logs** tab.

Click **Check my progress** to verify the objectives.



Task 3. Deploy a Genkit flow to Firebase

Firebase Genkit includes a plugin that helps you deploy your flows to Cloud Functions for Firebase. This task walks you through the process of deploying a default sample flow to Firebase.

Set up Firebase

1. To install **Firebase tools**, run:

```
npm install -g firebase-tools@14.0.1
```

The `npm install` command installs the Firebase Command Line Interface (CLI)

> **Note:** This version of the Firebase CLI is required to initialize the latest version of genkit in the later steps of this task.

2. Update the path, and verify the version of Firebase installed:

```
echo 'export PATH="$(npm config get prefix)/bin:$PATH"' >>
~/.bashrc
export PATH="$(npm config get prefix)/bin:$PATH"
echo "firebase version $(firebase --version)"
```

The path is updated to ensure that you run the firebase commands from the packages you just installed, instead of running them from preexisting installs that may exist in Cloud Shell.

3. In **Cloud Shell Terminal**, to create a directory for the new app and set the project and location variables, run the following commands:

```
mkdir ~/genkit-fire
cd ~/genkit-fire
export GCLOUD_PROJECT=Project ID
export GCLOUD_LOCATION=Lab Region
```

4. To authenticate the **Firebase SDK**, run the following command:

```
firebase login --no-localhost
```

   - To allow Firebase to collect CLI and Emulator Suite usage and error reporting information, enter **Y**, and press **Enter**.
   - Click on the URL provided to open a new browser window and follow the steps provided on the screen to generate the verification code.
   - Copy the verification code, and then, in **Cloud Shell**, paste it as the **authorization code**, and press **Enter**.

5. To initialize a new Genkit project, run the following command:

```
firebase init genkit
```

   - Select the **Use an existing project** option, and press **Enter**.
   - Select project `Firebase Project Id` as the default Firebase project for
   - To initialize functions, type **Y**, and press **Enter**.
   - To not use ESLint, type **n**, and press **Enter**.
   - To install dependencies with npm, type **Y**, and press **Enter**.
   - After the dependencies are installed, to install the Genkit CLI locally, select **Just this project**, and press **Enter**.
   - Select **Google Cloud Vertex AI** as your model provider, and press **Enter**.
   - After the NPM packages are installed, select **Set if unset** to update your tsconfig.json with suggested settings, and press **Enter**.
   - Select **Set if unset** to update your package.json with suggested settings, and press **Enter**.

- To generate a sample flow, type **Y**, and press **Enter**.

Set up your Genkit application

```
mv ~/genkit-fire/functions/src/genkit-sample.ts ~/genkit-fire/functions/src/index.ts
```

2. To change the default region for Vertex AI, run the following command:

```
sed -i 's/us-central1/Region/g' ~/genkit-fire/functions/src/index.ts
```

3. Update the Gemini model used in the sample application:

```
sed -i 's/gemini15Flash/gemini20Flash/g' ~/genkit-fire/functions/src/index.ts
```

> The latest genkit library used by the genkit-fire app supports the latest Gemini 2.0 Flash model *gemini20Flash*. Note that the genkit-intro app that you developed in the earlier lab tasks uses a previous genkit library version that only supports the stable model version *gemini20Flash001*. For more information, refer to the documentation on model versions and lifecycle.

open the `index.ts` file.

5. To define an authorization policy for the flow, in the `onCallGenkit()` method, uncomment the **authPolicy** parameter:

```
authPolicy: hasClaim('email_verified'),
```

To deploy your flow to Firebase Cloud Functions, you must wrap the flow in the `onCallGenkit()` method that enables you to quickly create a callable function in Firebase.

6. Add the `cors`, and `region` parameters to the `onCallGenkit()` function:

```
cors: true,
region: 'Lab Region',
```

> **Note:** This CORS setting allows any origin, which is designed to simplify the development of this lab. You will want a more restrictive CORS policy for production apps.

in Google AI which reqires an API key to be provided. Instead, the flow uses Gemini in Vertex AI.

8. Also, comment out these lines above the `genkit` initialization code:

```
//import { defineSecret } from "firebase-functions/params";
//const apiKey = defineSecret("GOOGLE_GENAI_API_KEY");

const ai = genkit({
    plugins: [
        ...
    ],
});
```

After the updates are made, the `onCallGenkit()` method invocation should look like this: (Comments have been removed from the snippet below)

```
export const menuSuggestion = onCallGenkit({
    // enforceAppCheck: true,
    authPolicy: hasClaim("email_verified"),

    // secrets: [apiKey],
```

```
}, menuSuggestionFlow);
```

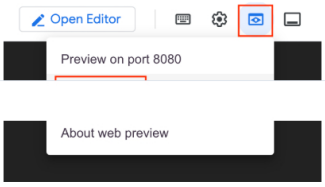
Test the flow in the Genkit Developer UI

1. Click **Open Terminal**, navigate to the `~/genkit-fire/functions` folder, and try the flow in the developer UI.

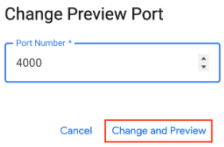
```
cd ~/genkit-fire/functions
npx genkit start -- npx tsx src/index.ts | tee -a flow_response.txt
```

Wait for the command to return the **Genkit Developer UI** URL in the output before continuing to the next step.

2. Click the **Web Preview** button, and click **Change port**.



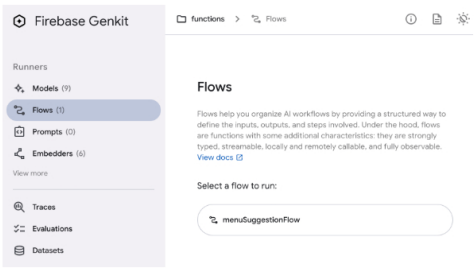
3. Change the port to `4000`, and click **Change and Preview**.



The **Genkit Developer UI** opens in your browser.

> The Developer UI might indicate a warning that it is waiting to connect to the Genkit flow server. Wait until the connection is established before proceeding.

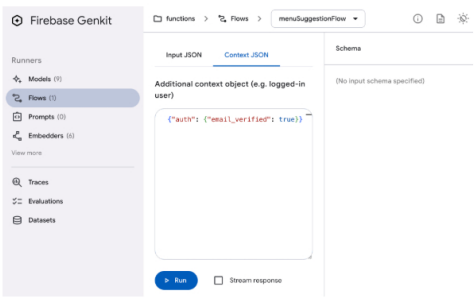
4. On the left side panel, click **Flows**. For the flow to run, click



5. On the **Input JSON** tab, use the following subject for the model:

```
"seafood"
```

6. Click the **Context JSON** tab, and use the following as a simulated auth object:



7. To test the flow, click **Run**.

   The output from the flow is displayed.

> **Note:** You might have to go back to the Google Cloud Console tab and accept the popup asking for authorization.

CTRL+C to exit.

9. For lab assessment purposes, run the following commands to move the file to the Cloud Storage Bucket named `GCS Bucket Name`.

```
gsutil -m cp -r ~/genkit-fire/functions/src/index.ts
gs://GCS Bucket Name
gsutil -m cp -r ~/genkit-fire/functions/flow_response.txt
gs://GCS Bucket Name
```

Deploy the flow to Firebase

1. Now, deploy the function to **Firebase** using the following command:

```
cd ~/genkit-fire
firebase deploy --only functions
```

Note: Re-run the command if you get the following error: **User code failed to**

2. If prompted to enter the number of days to keep container images, use the default and press Enter.

   You've now deployed the flow as a **Cloud Run function**, but you won't be able to access your deployed endpoint with curl or a similar tool, because of the flow's authorization policy. Continue to the next section to learn how to securely access the flow.

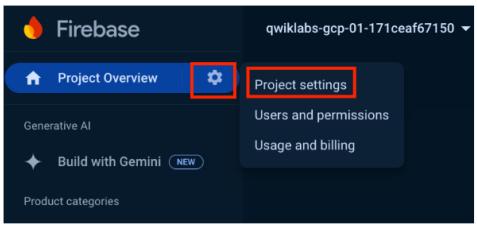
   The default sample flow uses the authorization policy:

```
authPolicy: hasClaim("email_verified")
```

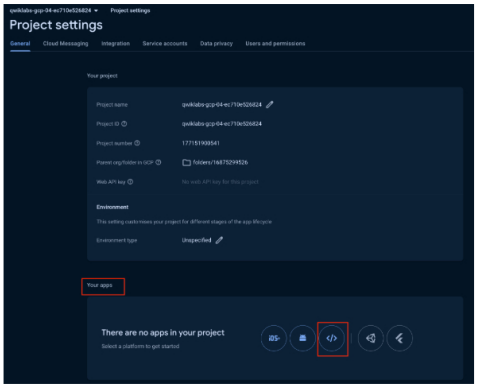
   The `onCallGenkit()` wrapper provided by the Firebase Functions library uses Firebase Auth to protect your flows. It has built-in support for the Cloud Functions for Firebase client SDKs.

Test the deployed flow

1. In a new tab of the same browser window, open the Firebase console (https://console.firebase.google.com/).
2. Select the project `Firebase project`.
3. If prompted, accept the **Firebase Terms and Conditions**.
4. In the Firebase console, click on the **Setting** icon next to **Project Overview**, and click **Project settings**.



5. In the **Your apps** section, to add a new web app, click the **Web App** (**</>**)



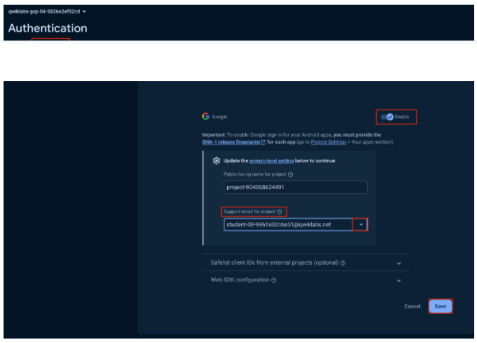
6. For **App nickname**, type `Firebase Web App Name`
7. Select **Also set up Firebase Hosting for this app**, and click **Register app**.

   as their defaults and click **Next** for both sections.

9. For the **Deploy to Firebase Hosting** section, click **Continue to Console**.
10. In the left panel, click **Build**, and select **Authentication**.
11. To begin setting up user authentication for the app, click **Get started**.

12. For **Sign-in providers**, click **Google**, then click **Enable**

13. For **Support email for project**, select the student email, then click **Save**.



14. To set up **Firebase Hosting**, return to **Cloud Shell** and run the following commands:

```
cd ~/genkit-fire
firebase init hosting
```

- For public directory, to keep the default of **public**, press **Enter**.
- To not set up automatic builds with GitHub, type **N**, and press **Enter**.

15. Click **Open editor**, navigate to the `~/genkit-fire/public` folder, and replace the code in `index.html` with the following:

```
<!doctype html>
<html>
  <head>
    <title>Genkit demo</title>
  </head>
  <body>
    <div id="signin" hidden>
      <button id="signinBtn">Sign in with Google</button>
    </div>
    <div id="callGenkit" hidden>
    Subject: <input type="text" id="subject" />
      <button id="suggestMenuItem">Suggest a menu
theme</button>
      <p id="menuItem"></p>
    </div>
    <script type="module">
      import { initializeApp } from
'https://www.gstatic.com/firebasejs/11.0.1/firebase-
app.js';
```

```
        onAuthStateChanged,
        GoogleAuthProvider,
        signInWithPopup,
      } from
'https://www.gstatic.com/firebasejs/11.0.1/firebase-
auth.js';
      import {
        getFunctions,
        httpsCallable,
      } from
'https://www.gstatic.com/firebasejs/11.0.1/firebase-
functions.js';

      const firebaseConfig = await
fetch('/__/firebase/init.json');
      initializeApp(await firebaseConfig.json());
      var functions = getFunctions();
      functions.region = 'Region';

      async function generateMenuItem() {
        const menuSuggestionFlow =
httpsCallable(functions,'menuSuggestion');
        const subject =
document.querySelector('#subject').value;
        const response = await menuSuggestionFlow(subject);
        document.querySelector('#menuItem').innerText =
```

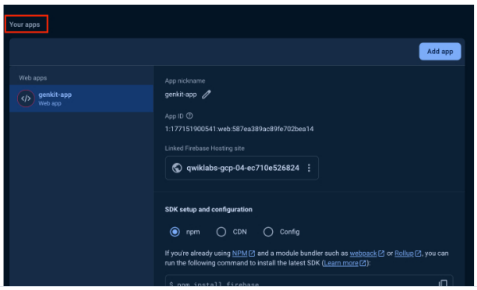
```
      function signIn() {
        signInWithPopup(getAuth(), new
GoogleAuthProvider());
      }


document.querySelector('#signinBtn').addEventListener('click'
signIn);

document.querySelector('#suggestMenuItem').addEventListener('
generateMenuItem);

      const signinEl = document.querySelector('#signin');
      const genkitEl =
document.querySelector('#callGenkit');

      onAuthStateChanged(getAuth(), (user) => {
        if (!user) {
          signinEl.hidden = false;
          genkitEl.hidden = true;
```

```
    } else {
      signinEl.hidden = true;
      genkitEl.hidden = false;
    }
  });
```

```
</html>
```

16. Return to the **Firebase Console**, click the **Settings** icon next to **Project Overview**, and then click **Project settings**.

17. Scroll down to the **Your apps** section, and for the large code block, click the copy button.



18. Navigate back to **Cloud Shell**, click on **Open Editor**, and navigate to the `~/genkit-fire/functions/src` folder.

19. Paste the copied code into the **index.ts** file after the import section and above the `genkit` initialization code.

20. Find the **initializeApp** line that was just copied in, and remove `const app =`.

    The new line should contain only `initializeApp(firebaseConfig);`.

21. Click **Open terminal**, and then, to add the `firebase` dependency to the `package.json` file, run the following commands:

```
cd ~/genkit-fire/functions
npm install firebase --save
```

22. Deploy the web app and Cloud Run function using the following command:

```
cd ~/genkit-fire
firebase deploy
```

23. To open the web app, in a new browser tab, enter the following URL:

```
https://PROJECT ID.web.app/
```

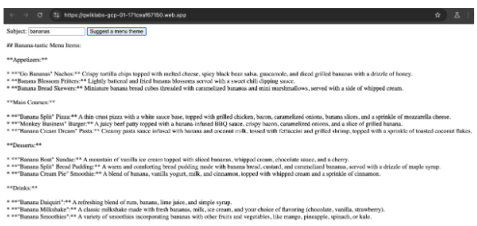
    The app requires you to login with a Google account, after which you can initiate endpoint requests.

24. On the web app page, click **Sign in with Google**, and select the Google account `Lab Username` and follow the screen instructions.

    The entry form is displayed.

    theme.

    After a short delay, the application returns the flow response, which may resemble this:



    Click **Check my progress** to verify the objectives.

Deploy the flow to Firebase

Check my progress

Task 4. Developing using Firebase Local Emulator Suite

Firebase offers a suite of emulators for local development, which you can use with **Genkit**.

To use Genkit with the **Firebase Emulator Suite**, start the Firebase emulators by running the following commands.

1. To use Genkit with the **Firebase Emulator Suite**, in Cloud Shell, run the following commands:

```
cd ~/genkit-fire
GENKIT_ENV=dev firebase emulators:start --inspect-functions
| tee -a response.txt
```

This will run your code in the emulator and run the **Genkit framework** in

The emulators' UI is hosted on localhost port 4000.

2. Click the **Web Preview** button, and click **Change port**.
3. Change the port to `4000`, and click **Change and Preview**.

The emulators UI is presented.

4. Keep the emulators running in this terminal tab, and open a new tab using the **+** button in **Cloud Shell**.
5. To launch the Genkit Developer UI and run it against the emulated code, run the following commands:

```
cd ~/genkit-fire/functions
npx genkit start -- npx tsx src/index.ts --attach
http://localhost:3100 --port 4001 | tee -a response.txt
```

The Genkit Developer UI will be hosted on port 4001. Instead of calling a Cloud Run function, the developer UI uses the emulator.

7. Change the port to `4001`, and click **Change and Preview**.

The Genkit developer UI opens in a new tab, and you can make calls against the emulator.

8. Return to the **Cloud Shell terminal** and press **CTRL+C** to exit.
9. Run the following commands to move the file to the Cloud Storage Bucket named `GCS Bucket Name`.

```
gsutil -m cp -r ~/genkit-fire/response.txt gs://GCS Bucket
Name
```

Click **Check my progress** to verify the objectives.

Develop web app using Firebase Local Emulator

Check my progress

Congratulations!

You have successfully created a Genkit project and deployed it both to Google Cloud Run and Firebase. Additionally, you tested the application through the Genkit development UI and the CLI interface.

Manual Last Updated February 14, 2025

Lab Last Tested February 14, 2025